



Лекція 1 - Основи Java

Примітивні типи даних

Цілочисельні

- **byte**
- **short**
- **int**
- **long**

Дробові

- **float**
- **double**

Символьні

- **char**

Логічні

- **boolean**

Приклад:

int age = 25;

double price = 19.99;

boolean active = true;

char letter = 'A';

Класи-обгортки примітивних типів

boolean **-> Boolean**

int **-> Integer**

byte **-> Byte**

ТОЩО

Всі об'єкти-обгортки – незмінні (immutable).

Autoboxing and Unboxing:

```
int a = 5;
```

```
Integer b = a;      // autoboxing
```

```
int c = b;      // unboxing
```

Створення об'єктів

```
Long value1 = new Long(41);      // так краще не робити (deprecated)
```

```
Long value2 = Long.valueOf(42);
```

String

String - незмінний об'єкт.

```
String name = "Bilbo"
```

```
name = name + " Baggins" // новий об'єкт
```

Як працює **String Pool**:

```
String a = "Java";
```

```
String b = "Java";
```

```
String c = new String("Java");
```

```
a == b // true — тому що вони з string pool
```

```
a == c // false — різні об'єкти
```

```
a.equals(c) // true — значення однакове
```

String

Корисні методи **String**:

```
name.isEmpty();    // рядок "" – порожній  
name.isBlank();    // "", " " – також порожній (Java 11)  
name.length();  
name.toUpperCase();  
name.contains("Java");  
name.strip();       // кращий варіант trim(), підтримує Unicode  
name.repeat(3);     // Java 11
```

StringBuilder

Коли String не підходить — **StringBuilder**

```
StringBuilder sb = new StringBuilder();  
    for (int i = 0; i < 1000; i++) {  
        sb.append(i).append(", ");  
    }  
    System.out.println(sb);
```

Multi-line Strings + Formatted

```
String content = """
    {
        "id": %d,
        "name": "%s"
    }"""
    .formatted(1, "John");
System.out.println(content);
```

Output:

```
{
    "id": 1,
    "name": "John Smith"
}
```

if / else

```
if (age >= 18) {  
    System.out.println("Доступ дозволено");  
} else {  
    System.out.println("Доступ заборонено");  
}
```

Рекомендація: використовуйте фігурні дужки “{ }” завжди

Тернарний оператор:

```
String status = age >= 18 ? "Дорослий" : "Неповнолітній";
```


Switch (класичний)

```
switch (day) {  
    case 1:  
        System.out.println("Понеділок");  
        break;  
    case 2:  
        System.out.println("Вівторок");  
        break;  
    default:  
        System.out.println("Невідомий день");  
}
```

- Кожен case треба завершувати break, інакше виконання "провалиться" далі.
- Не повертає значення – це просто блок коду.
- Перевантажений зайвим синтаксисом.

Switch (сучасний)

```
String dayName = switch (day) {  
    case 1 -> "Понеділок";  
    case 2 -> "Вівторок";  
    case 6, 7 -> "Вихідний";  
    default -> "Невідомий день";  
};
```

Переваги нового синтаксису:

- Немає break – кожен case завершується автоматично.
- Працює як вираз – можна присвоїти результат у змінну.
- Може мати кілька значень в одному case
- Зручніше, безпечніше, коротше.

Масиви

Масиви – це об'єкти, які містять послідовність елементів (об'єктів або примітивних типів).

Оголошення масиву

```
long[] myArray = new long[]{7, 6, 3, 5, 7, 3, 2, 4};
```

Доступ – через індекс, починаючи з 0

Звичайний цикл for

```
for (int i = 0; i < myArray.length; i++) {  
    System.out.println(myArray[i]);  
}
```

Цикл foreach

```
for (long val : myArray) {  
    System.out.println(val);  
}
```

Цикл while

```
while (i < 5) {  
    i++;  
}
```

break — перериває цикл повністю
continue — переходить до наступної ітерації

Класи, об'єкти та об'єктні посилання

Клас у Java – це певний опис типу.

Об'єкт являє собою екземпляр класу.

Класи і об'єкти володіють **методами** і **атрибутами**.

Доступ до об'єктів і виклик їх методів здійснюється за допомогою **об'єктних посилань**.

- Посилання може не посилатися ні на який об'єкт — у такому випадку це пусте (**null**) посилання;
- Всі посилання жорстко **типізовані**.

Дані **примітивних типів** не є посиланнями.

Атрибути та методи класів

Атрибути класу визначають із яких даних будуть складатися об'єкти даного класу.

- Атрибути можуть бути посиланнями на інші об'єкти або простими типами.

Методи класу визначають поведінку об'єкту цього класу.

Методи та атрибути можуть мати наступні **області видимості**:

- **public** — доступ без будь-яких обмежень;
- **private** — доступ дозволений лише із даного класу;
- **protected** — доступ дозволений із даного класу та з усіх класів-нащадків, а також із усіх класів даного пакету.
- *"friendly"* — доступ дозволений із усіх класів даного пакету.

Конструктори

Конструктор – це спеціальний метод класу, який викликається при створенні об'єкту.

Конструктор без параметрів називається **конструктором за замовчуванням** (default constructor).

Якщо клас не містить жодного конструктору, то генерується пустий конструктор за замовчуванням. Якщо клас містить хоча б один конструктор, то конструктор за замовчуванням не генерується.

Якщо у батьківського класу поточного класу є конструктор без параметрів, то він викликатиметься перед викликом конструктору дочірнього автоматично.

Якщо у батьківського класу заданий лише конструктор з параметрами – він може бути викликаний лише на початку конструктора дочірнього класу.

```
public SomeClass(parentArgs, newArgs){  
    super(parentArgs);  
    ...  
}
```

Getters and Setters

Область видимості змінних атрибутів класу завжди краще робити **private** або **protected**, а для доступу до цих полів застосовувати get- та set-методи.

```
private int age;  
public int getAge(){  
    return age;  
}  
public void setAge(int age){  
    this.age = age;  
}
```


Демонстрація роботи зі статичним атрибутом

```
public class Counter {  
  
    private static int cnt = 0; // статичне поле  
  
    public Counter(){  
        cnt++; // збільшує значення cnt при створенні  
               // кожного нового об'єкту цього класу  
    }  
  
    public static void main(String args[]) {  
        Counter c1 = new Counter(); // cnt дорівнюватиме 1  
        Counter c2 = new Counter(); // cnt дорівнюватиме 2  
        System.out.println(cnt);  
    }  
}
```

Особливості статичних методів

Статичні методи **не прив'язані** до конкретного **об'єкту** класу.

При виклику статичного методу перед ним необхідно вказати ім'я класу.

```
class StringUtil {  
    public static void countSpaces(String str) {  
        // статичний метод  
    }  
}
```

```
StringUtil.countSpaces("рядок з пробілами");
```

Всередині статичного методу не можна звертатися до **нестатичних полів і методів** класу без посилання на об'єкт.

Модифікатор доступу *final*

Застосування модифікатору до полів, змінних і параметрів забороняє їх зміну .

Константи зазвичай оголошуються як **static final**.

```
public static final double PI = 3.14;
```

Увага: у разі застосування модифікатору **final** при описі об'єктних посилань незмінним буде лише саме посилання. Однак це не означає, що поля об'єкту будуть незмінними.

```
static final int[] mas = {1, 2, 3};  
mas[2] = 100; // допустимо  
mas = new int[]{5, 6, 7}; // недопустимо
```

final-метод не може бути перевизначений у класах-наслідниках.
final-клас не може бути унаслідований.

Enums

Enumeration – це клас, кількість екземплярів якого суворо обмежена.

```
public enum Day {
```

```
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,  
    SUNDAY;
```

```
    @Override  
    public String toString() {  
        return name().substring(0, 1) +  
            name().substring(1).toLowerCase();  
    }  
}
```

Метод *name()* повертає ім'я об'єкту.

Метод *ordinal()* повертає порядковий номер об'єкту.

Enums

Посилання на об'єкт enum.

```
Day day = Day.WEDNESDAY;
```

Методи *values()* та *valueOf()*

```
for (Day day: Day.values()) {  
    System.out.println(day);  
}
```

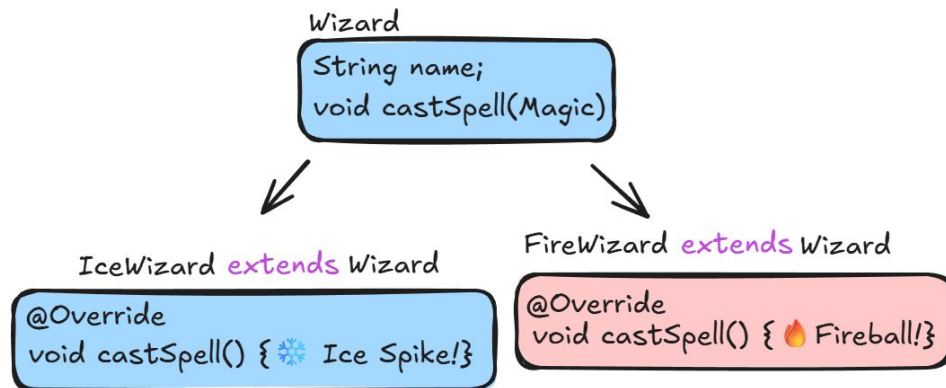
```
Day day = Day.valueOf("WEDNESDAY");
```

Наслідування класів

Наслідування дозволяє будувати нові класи на основі існуючих, додаючи в них нові можливості або перевизначаючи існуючі.

Не можна наслідувати більше одного об'єкту.

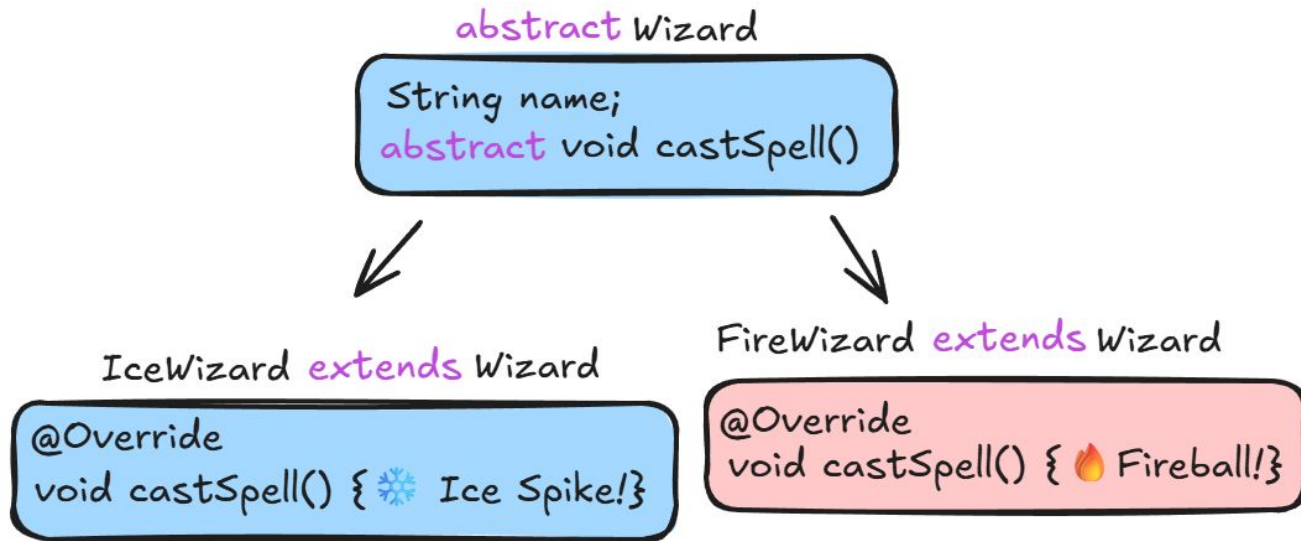
На об'єкти дочірнього класу можна посилатися за посиланням батьківського класу (downcasting).



```
Wizard w = new IceWizard();
w.castSpell();
```

Абстрактні класи

Абстрактний клас – це клас, від якого ведуть наслідування інші класи, а об'єктів цього класу не існує.



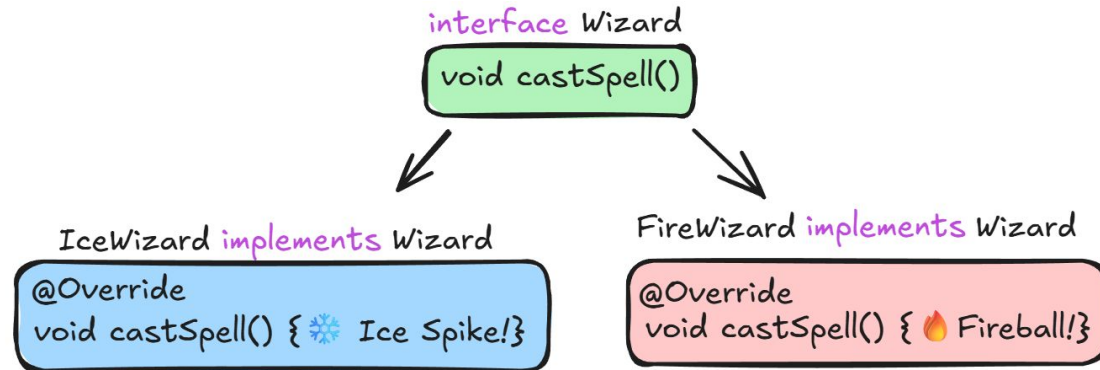
Інтерфейси

Інтерфейс – це клас, в якому всі методи є **public** та **abstract** (окрім **default**).

Поля інтерфейсу за замовчуванням є **final**, **public** та **static**.

Дані модифікатори рекомендується не вказувати.

Один клас може реалізовувати декілька інтерфейсів.



Анонімні класи

Анонімні класи — це класи без імені.

Опис анонімного класу та **створення об'єкту здійснюється одночасно** — це дозволяє скоротити код.

```
interface Wizard {  
    void castSpell();  
}  
  
...  
Wizard lightningWizard = new Wizard() {  
    @Override  
    public void castSpell() {  
        System.out.println("⚡Chain Lightning!");  
    }  
};
```

Типи, що налаштовуються (Generics)

Generics дозволяють розробникам створювати структури даних з відкладеним вибором типів.

```
class Box<T> {  
    private T value;  
    public Box(T value) {  
        this.value = value;  
    }  
    public T get() {  
        return value;  
    }  
  
    public void set(T value) {  
        this.value = value;  
    }  
}
```

```
Box<String> messageBox = new Box<>("Hello, world!");  
Box<Integer> numberBox = new Box<>(123);
```

Лямбда-вирази

```
public class LambdaTest {  
    interface Operation<T> {  
        T perform(T p1, T p2);  
    }  
  
    public static void main(String[] args) {  
        Operation<Integer> add = (v1, v2) -> v1 + v2;  
        Operation<Integer> sub = (v1, v2) -> v1 - v2;  
  
        System.out.println(add.perform(3, 5));  
        System.out.println(sub.perform(3, 5));  
    }  
}
```

Функціональні інтерфейси

Функціональні інтерфейси можна позначити анотацією **@FunctionalInterface**

Стандартні функціональні інтерфейси

Consumer – виконати з параметром.

Runnable – виконати без параметра.

Function – прийняти параметр та повернути значення.

Predicate – прийняти параметр і повернути **boolean**.

Анотації (метадані)

Анотації дозволяють включати в Java програму довільну інформацію (метадані).

Ця інформація може використовуватися:

- компілятором (наприклад, **@Override**);
- іншими програмними компонентами, що працюють з вашим класом (під час виконання).

В Java є можливість створювати свої анотації, використовувати їх в класах і обробляти власними програмними компонентами.

Приклад використання анотацій із Spring Framework:

@Transactional

```
public class AutoService {
```

```
    @Transactional(readOnly=true)
```

```
    public Auto load(long id) {
```

```
        // Виконується завантаження об'єкту
```

```
    }
```

```
}
```

Клас Object

Object – базовий клас в Java. Від нього наслідуються всі інші класи.

Деякі методи класу Object:

- **clone()** – створює та повертає копію цього об'єкту;
- **equals(Object obj)** – вказує на те, що цей об'єкт у деякому сенсі дорівнює іншому;
- **finalize()** – викликається «прибиральником сміття» (garbage collector) перед видаленням об'єкту;
- **hashCode()** – повертає хеш-код об'єкту;
- **toString()** – повертає рядкове представлення об'єкту;

Метод `boolean equals(Object obj)`

Метод `equals` слід перевизначити тоді, коли для класу існує поняття логічної еквівалентності, яке не співпадає з тотожністю об'єктів.

Метод `equals` повинен задовольняти наступні вимоги:

- **Рефлексивність**
 - `a.equals(a)` завжди повинно бути **true**.
- **Симетричність**
 - якщо `a.equals(b) == true`, то і `b.equals(a)` повинно бути **true**
- **Транзитивність**
 - якщо `a.equals(b) == true`, `b.equals(c) == true` то і `a.equals(c)` повинно бути **true**
- **Несуперечливість**
 - `a.equals(null)` завжди повинно повертати **false**.

Метод equals(Object obj). Пример

```
public class Point {  
    private final int x;  
    private final int y;  
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }  
  
    public boolean equals(Object obj){  
        if (obj == this){  
            return true;  
        }  
        if (!(obj instanceof Point)){  
            return false;  
        }  
        Point p = (Point)obj;  
        return p.x == x && p.y == y;  
    }  
}
```


`equals()` vs. `==`

```
String a="abc";  
String b="abc";  
String c = new  
    StringBuilder("a").append("bc").toString();
```

```
System.out.println(a == b);  
System.out.println(a.equals(b));  
System.out.println(a == c);  
System.out.println(a.equals(c));
```

Метод hashCode()

Метод **int hashCode()** використовується при створенні хеш-таблиць (HashSet, HashMap тощо).

Вимоги до методу:

- він має бути перевизначений в **кожному класі**, де перевизначений метод **equals()**;
- **при декількох зверненнях** до об'єкту метод повинен повертати **одне і те саме число**;
- якщо два об'єкти рівні за результатами методу **equals()**, їх хеш-коди також повинні бути рівні;
- якщо два об'єкти не рівні за **equals()**, вони не обов'язково повинні мати різний хеш-код. Але це дуже бажано.

```
public int hashCode(){  
    int result = 17;  
    result = 37 * result + x;  
    result = 37 * result + y;  
    return result;  
}
```

Метод toString()

Метод **toString()** повертає рядкове представлення об'єкту.

В класі `Object` він формує рядок як

```
getClass().getName() + '@' + Integer.toHexString(hashCode()) = Person@3f99bd52
```

Цей метод викликається **при конкатенації рядків**, якщо вказується посилання на об'єкт, а також **при виведенні об'єкту у консолі**.

```
public class Point{
    private int x, y;

    public Point(int x, int y) { this.x = x; this.y = y; }

    public String toString(){
        return "(" + x + ", " + y + ")";
    }

    public static void main(String[] args){
        System.out.println(new Point(1, 3)) // Буде виведено (1, 3)
        // Дорівнюватиме "Point (2, 3)"
        String str = "Point " + new Point(2, 3);
    }
}
```

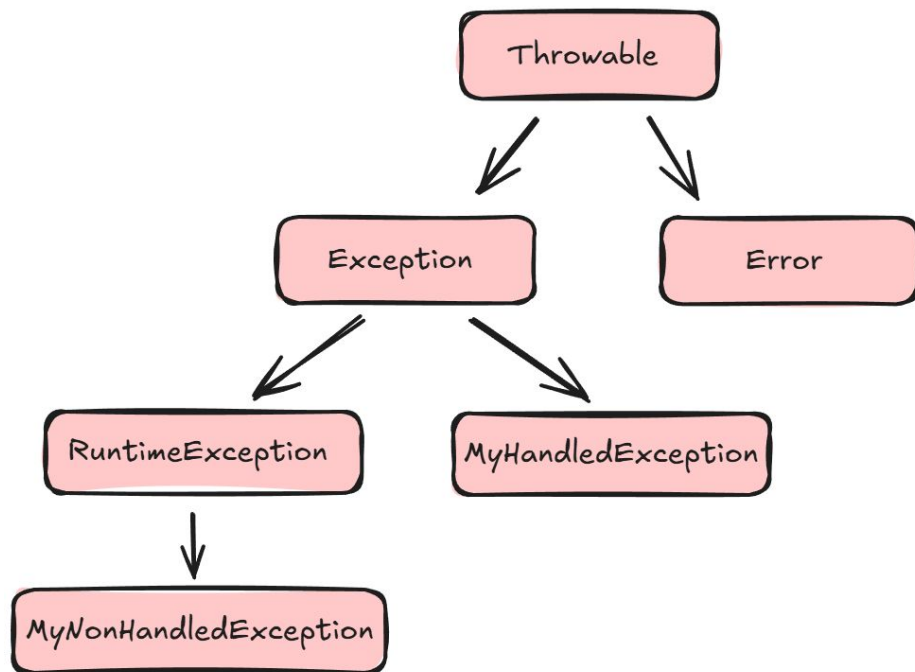
Exceptions

Throwable — базовий клас для всіх виключних ситуацій.

Error — базовий клас для виключних ситуацій, що пов'язані зі збоями в роботі віртуальної машини Java.

Exception — базовий клас для всіх тих виключень, з якими ми маємо справу в програмах.

RuntimeException — помилка часу виконання. Її обробка не обов'язкова для компілятора.



Генерація виключень

Два варіанти генерації виключень – **автоматична** і **програмна**.

Автоматична генерація виключення відбувається, якщо Java-машина виявляє деяку помилку, наприклад, ділення на нуль, помилку приведення типів або звернення до полів і методів об'єкту за об'єктним посиланням, значення якого null.

```
"abc".equals(str) // Такий варіант порівняння рядків безпечний,  
                  // оскільки при str == null вираз поверне false
```

```
str.equals("abc") // при використанні такого варіанту  
                  // якщо str == null виникатиме NullPointerException
```

Програмна генерація виключень

Програмна генерація виключень здійснюється за допомогою оператора **throw**:

```
throw new IllegalArgumentException("Параметр повинен бути  
позитивним числом");
```

При цьому, якщо виключення не обробляється в даному методі, в сигнатурі методу повинна стояти директива **throws**.

```
public InputStream getFileStream() throws IOException {  
    return new FileInputStream("dummy.txt");  
}
```

Обробка виключень

Виключна ситуація повинна бути оброблена належним чином.

```
try {  
    ...  
} catch (SomeException1 e) {  
    ...  
} catch (SomeException2 e) {  
    ...  
} finally {  
    ...  
}
```

В іншому випадку програма буде завершена достроково.

Приклад обробки виключення

```
try {  
    inputFile("data.txt");  
    calculate();  
} catch (FileNotFoundException e) {  
    System.out.println("Файл data.txt не знайдено");  
} catch (IOException e) {  
    System.out.println(e.getMessage());  
} catch (Exception e) {  
    log.error("General error processing file", e);  
}
```


Створення власного виключення

```
public class MyException extends Exception {  
    public MyException() {};  
    public MyException(String msg) {  
        super(msg);  
    };  
}
```

Слід відзначити, що в даному випадку створено виключення, що обов'язкове для перехоплення. Якщо потрібно створити виключення, яке не обов'язкове для перехоплення – потрібно унаслідувати його від **RuntimeException**.

Стандартні виключення часу виконання

IllegalArgumentException

Неправильне значення параметру

IllegalStateException

Стан об'єкту неприйнятний для виклику методу

NullPointerException

Значення параметру дорівнює **null**, що заборонено

IndexOutOfBoundsException

Значення параметру, що задає індекс, виходить за межі діапазону

ConcurrentModificationException

Виявлена паралельна модифікація об'єкту із різних потоків, що заборонено

UnsupportedOperationException

Об'єкт не має підтримки вказаного методу

Використовуйте виключення лише у виключних ситуаціях

Ніколи не робіть так:

```
try{  
    int i= 0;  
    while(true)  
        a[i++].f();  
}catch(ArrayIndexOutOfBoundsException e){}
```

Правильно так:

```
for (int i = 0; i < a.length; i++){  
    a[i].f();  
}
```

Не ігноруйте виключень

У разі виникнення виключення частина коду не буде виконана, а ми цього і не помітимо

```
try{  
    ...  
} catch(Exception e){  
    // Повинен бути щонайменше коментар!  
}
```